

TRAVIS TINCHER

Senior Software Engineer — Ruby on Rails · full stack · AI agents

tincher.travis@gmail.com · (903) 574-1520 · linkedin.com/in/travis-tincher · github.com/trtincher · Philadelphia, PA (Eastern) · remote

SUMMARY

Full-stack engineer with over six years of experience building and maintaining Ruby on Rails servers and React TypeScript clients, with PostgreSQL behind most of it. Shipped 3rd party and LLM integrations into regulated workflows in mortgage, estate settlement, and healthcare. Spent the past year building and operating an agent-driven development system with human review gates, which is where most of my understanding of how to verify AI output comes from.

SELECTED WORK — AUTONOMOUS AGENT DEV SYSTEM · 2025 – PRESENT

- Built and operate a two-mode agent system for shipping software. In live mode an orchestrator session plans the work, briefs parallel worker agents in isolated git worktrees, chooses the model for each task, and lands the results as a stacked set of pull requests with one human review gate. In background mode a queue-driven runner executes one headless LLM session per task from a pinned checkout for overnight and batch work.
- Over twelve months of continuous operation it has run about 3,500 tasks and merged 640+ pull requests across multiple repos, most of them fully autonomously with human review at strategic gates, and a multiplayer game beta shipped through it.
- Quality comes from checks the system runs on itself. I cut the test suites to a small budget per module after coverage chasing ate a third of one agent's run, and moved that weight onto verification per task. Every task carries its own verify criteria that have to pass before its commit lands, and after UI work the orchestrating agent opens the result in a real browser tab at phone width, clicks through the flows, and judges its own screenshots before I look, because headless checks had let broken screens through. That shift has raised the quality of what reaches me more than anything else, and each run's token cost rolls into a weekly review of which model tier is worth its price.

SKILLS

- **Backend:** Ruby, Rails 5–7, PostgreSQL, SQL, Active Job / SQS, aasm, REST, GraphQL, RSpec, Node.js / NestJS / TypeScript
- **Frontend:** TypeScript / JavaScript, React, HTML, CSS, Tailwind, Rails views, accessibility, design systems
- **AI agents:** orchestrator / worker patterns, worktree isolation, per-task model routing, Claude Code and headless Claude pipelines, agent-driven headed browser verification, OpenAI API, Textract, prompt engineering, agent evaluation
- **Deployment and quality:** AWS (ECS, RDS, S3, SQS, Textract), Terraform, GitHub Actions CI/CD, Docker, Sentry, automated testing, code review, feature-flag deployments

EXPERIENCE

Deep Seas Development — Ruby on Rails Engineer (contract)

Jan 2025 – May 2025, Apr 2026 – Present · remote

- Upgraded two consumer-facing legacy Rails applications from Ruby 2 / Rails 5 to Ruby 3 / Rails 7, adding test coverage, structured error logging, and documentation so the upgrades shipped without regressions.
- Maintain NestJS / TypeScript services on PostgreSQL alongside the Rails work; GitHub Actions CI/CD, AWS via Terraform.

Artisight — Senior Frontend Software Engineer

May 2025 – Jan 2026 · remote

- Built clinician-facing React / TypeScript apps for an AI smart-hospital platform (virtual nursing and observation console, admin and configuration tooling, patient-room bedside interface) deployed at NYU Langone, Johns Hopkins, Mayo Clinic and 30+ health systems.
- Shipped real-time features over WebSocket and WebRTC (two-way audio/video, live sensor and location events) and surfaced AI alerts and predictions to nurses with explicit uncertainty and failure states so they could tell a confident prediction from a weak one.
- Owned frontend testing and release verification (Jest, introduced Playwright, browser checks before and after release in live hospitals); built to the design system with accessibility as a hard requirement.

Alix — Founding Full Stack Engineer (estate-settlement platform)

May 2023 – Dec 2024 · remote

- Designed and shipped an LLM document pipeline (S3 + Textract + OpenAI) that classifies and auto-sorts hundreds of scanned estate documents per client, with batching for production volume and a prompt-engineered table-of-contents output. Cut manual processing 80%, about 15 hours per client.
- Moved the team from bottlenecked manual QA to weekly feature-flagged releases: integrated the flag system, rebuilt the Jira automation, taught the process. Defects fell about 70% and hotfixes became rare.
- Built the React / TypeScript client and admin portal from scratch with a design system; GraphQL codegen and form generators cut interface and admin-form work 65% and 90%.

JMJ Financial

Feb 2021 – Jan 2023 · remote

Ruby on Rails Engineer, backend lead · Feb 2022 – Jan 2023

- Architected the Rails / PostgreSQL backend for a mortgage platform on AWS: ECS web and worker services, Active Job on SQS, RDS in high availability, encrypted S3 via Active Storage, Sentry and CloudWatch.
- Built the Mortgage Disclosures feature end to end: Encompass and HelloSign integrations, an aasm state machine per document, webhooks for the event-driven approval flow, and Rails views for loan officers. A multi-hour manual process became a background job with two review touchpoints (~60% less manual overhead).
- Designed the REST APIs for three client portals; shipped Auth.net and PDFtron integrations; led a project with an external contractor.

Full Stack Ruby on Rails Engineer · Feb 2021 – Jan 2022

- Built the Rails API layer and page-data hydration for the React client and internal Rails pages; shipped the digital loan application (Form 1003) end to end; owned CI/CD and pre-launch QA.

EDUCATION

General Assembly, Software Engineering, 2020 · Emory University, M.T.S., 2015 · Texas A&M, B.A. Philosophy, 2012